

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I _J BHARATH 192511234_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Answer: State = (x, y), where x = water in the 4-gallon jug, y = water in the 3-gallon jug.

- Step 0 (Start): (0, 0)
- Step 1: Fill the 4-gallon jug $\rightarrow$ (4, 0)
- Step 2: Pour 4-gallon jug into 3-gallon jug $\rightarrow$ (1, 3)
- Step 3: Empty the 3-gallon jug $\rightarrow$ (1, 0)
- Step 4: Pour 4-gallon jug (1 gal) into 3-gallon jug $\rightarrow$ (0, 1)
- Step 5: Fill the 4-gallon jug $\rightarrow$ (4, 1)
- Step 6: Pour 4-gallon jug into 3-gallon jug (only 2 gal fit) $\rightarrow$ (2, 3) — GOAL

At Step 6 the 4-gallon jug contains exactly 2 gallons — goal state reached in 6 actions.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

- i. What are the percept's for this agent?

Answer: Camera/stereo-vision images of terrain and rock/soil samples; spectrometer and chemical-composition sensor readings; GPS and wheel-odometry data; accelerometer/gyroscope readings (tilt, orientation); obstacle-detection data (lidar/radar); temperature and solar-panel power-level readings; uplink commands and data relayed from Earth/orbiter.

- ii. Characterize the operating environment.

Answer: Partially observable (limited sensor range, dust can obscure terrain); stochastic (wheel slippage, weather, terrain response are unpredictable); sequential (actions affect future states); dynamic (conditions can change during long communication delays); continuous (position, speed, sensor values are continuous); single-agent (unless coordinating with a companion rover); unknown (exact terrain/geology not known in advance).

- iii. What are the actions the agent can take?

Answer: Move forward/backward/turn; extract or drill soil and rock samples; operate the robotic arm; capture images; run onboard chemical/spectrometric analysis; transmit data to orbiter/Earth; adjust solar-panel orientation; detect and avoid obstacles.

- iv. How can one evaluate the performance of the agent?

Answer: Distance safely traveled; quantity and scientific value of samples analyzed; accuracy/quality of collected data; energy efficiency; avoidance of

damage or becoming stuck; mission time efficiency; successful data-transmission rate.

- v. What sort of agent architecture do you think is most suitable for this agent? Why?

Answer: A model-based, goal-based (utility-based) agent. Because the environment is partially observable, the rover needs an internal model (map, sensor history) to track what it cannot currently perceive. Because it must reach specific goals (target sample sites) while trading off safety, time, and energy, it needs goal/utility-based reasoning rather than simple reflex behaviour. In practice it also has a learning component (e.g. autonomous terrain-hazard learning), making it a hybrid learning + model-based + goal-based agent.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

Answer: Problem formulation for the 8-Queens problem (incremental formulation):

- State: Any arrangement of 0 to 8 queens on the board, with at most one queen per column.
- Initial state: No queens on the board.
- Successor function (Actions): Add a queen to the left-most empty column, in any row not attacked by an existing queen (same row or diagonal excluded). This reduces the search space to just 2,057 states, versus the far larger complete-state formulation.
- Goal test: All 8 queens are placed with no two sharing a row, column, or diagonal.
- Path cost: Constant (e.g. 1 per placement) — only the final configuration matters, so this is more naturally a Constraint Satisfaction Problem (CSP), typically solved with backtracking or local search (min-conflicts, hill-climbing) rather than plain BFS/DFS.

Note: the configuration described in the question (queen in column 1 sharing a diagonal with the queen in the last column) is a non-goal state — it shows why the goal test must check diagonals, not just rows and columns.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

Answer: Type of agent: a goal-based / utility-based problem-solving agent. The customer's goal is simply “reach the destination,” but multiple cab types (mini, micro, sedan, shared, prime) satisfy that goal differently in cost, comfort, and time — so the agent must search among alternatives and select the option that maximizes the user's utility, not merely find any path to the goal.

Pseudocode:

```
function BOOK_CAB(pickup_location, destination, user_preferences):
```

```

goal <- destination
available_cabs <- GET_AVAILABLE_CAB_TYPES(pickup_location)
                // e.g. mini, micro, sedan, shared, prime

options <- []
for each cab_type in available_cabs:
    fare <- ESTIMATE_FARE(cab_type, pickup_location, destination)
    eta  <- ESTIMATE_ARRIVAL_TIME(cab_type, pickup_location)
    capacity <- GET_CAPACITY(cab_type)
    options.append({cab_type, fare, eta, capacity})

// filter by hard constraints (budget, seats needed)
valid_options <- FILTER(options, user_preferences.max_fare,
                        user_preferences.passengers)

if valid_options is empty:
    return "No cabs match your requirements"

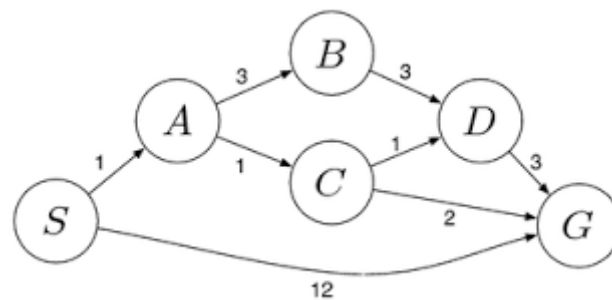
// rank remaining options by utility function
best_cab <- ARGMAX(valid_options,
                    UTILITY(fare, eta, comfort_level,
user_preferences.weights))

CONFIRM_BOOKING(best_cab, pickup_location, destination)
return best_cab

```

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse *S* to the destination warehouse *G* using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Answer — UCS Algorithm Steps:

- Insert the start node *S* into the frontier (a priority queue) with cumulative cost 0.
- Repeatedly remove the node with the LOWEST cumulative cost $g(n)$ from the frontier.
- If the removed node is the goal *G*, stop — the path found is optimal.
- Otherwise expand it: for each neighbour, new cost = cost(current) + edge weight.
- If a neighbour is new, or is reached via a cheaper cost than previously recorded, add/update it in the frontier.

- Repeat until G is popped from the frontier. UCS guarantees the optimal (least-cost) path because it always expands the cheapest frontier node first — unlike BFS, which only counts number of hops.

Frontier expansion table (fill in exact node labels / edge weights from the diagram above):

- Step 1 — Expand S — Path: S — Cost: 0
- Step 2 — Expand lowest-cost neighbour of S — Path: S → ... — Cost: (edge weight)
- Step 3 — Expand next lowest-cost frontier node — Path: ... — Cost: ...
- Step n — G reached — Path: S → ... → G — Cost: total = sum of edge weights on that path

Compare the total cost of every path from S to G and select the path with the minimum total cost as the final answer — even if it has more hops than a shorter-looking path. That comparison (total edge-weight sum per path, not hop count) is the key concept UCS is testing.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

		engineering relevance			
Problem Understandin g	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem